

HSD Arena - Games WebSocket & HTTP API (Frontend Rehberi)

Bu doküman **frontend geliştiriciler** için hazırlanmıştır. Games endpoint'lerini kullanarak real-time quiz oyunu nasıl entegre edilir detaylıca açıklanmıştır.

1. Bağlantı Bilgileri

- **WebSocket URL (Development):** ws://localhost:3000/games/ws
 - **WebSocket URL (Production):** wss://your-domain.com/games/ws
 - **HTTP Base URL:** http://localhost:3000 (development)
-

2. HTTP Endpoints (Kurulum)

2.1. POST /games - Oyun Oluştur (Admin/Host)

İstek (Request):

JSON

POST /games

Content-Type: application/json

```
{  
  "quizId": "uuid-of-quiz"  
}
```

Yanıt (Response):

JSON

```
{
```

```
"success": true,  
"gameId": "generated-game-id",  
"pin": "482910", // 6 haneli PIN  
"qrUrl": "https://...", // QR kod URL'i  
"mode": "PERSONAL" // veya "STAGE"  
}
```

2.2. GET /games/:pin - Oyun Özeti Al

İstek (Request):

Plaintext

GET /games/482910

Yanıt (Response):

JSON

```
{  
  "success": true,  
  "status": "FINISHED",  
  "totalPlayers": 150,  
  "winner": "Oyuncu1",  
  "finalScores": [  
    { "nickname": "Oyuncu1", "score": 15000 },  
    { "nickname": "Oyuncu2", "score": 14500 }  
  ]  
}
```

2.3. DELETE /games/:pin - Oyunu Sil

İstek (Request):

Plaintext

DELETE /games/482910

Yanıt (Response):

JSON

```
{  
  "success": true  
}
```

3. WebSocket Events (Client → Server)

Frontend'in Backend'e göndereceği olaylar.

3.1. JOIN_ROOM - Lobiye Katıl

- **Ne Zaman:** Oyuncu oyuna katılırken.
- **Gönderilecek Veri:**

```
JSON  
{  
  "type": "JOIN_ROOM",  
  "data": {  
    "pin": "482910",  
    "nickname": "Ahmet"  
  }  
}
```

- **Notlar:**
 - Nickname çakışması varsa backend otomatik #1, #2 ekler.
 - IP ban kontrolü yapılır.

3.2. KICK_PLAYER - Oyuncu At (Sadece Host)

- **Ne Zaman:** Host bir oyuncuyu atmak istediğinde.
- **Gönderilecek Veri:**

```
JSON  
{
```

```
"type": "KICK_PLAYER",
"data": {
  "socketId": "socket-id-123",
  "ban": true // true ise IP ban edilir
}
```

3.3. START_GAME - Oyunu Başlat (Sadece Host)

- **Ne Zaman:** Host oyunu başlatmak istediğinde.
- **Gönderilecek Veri:**

```
JSON
{
  "type": "START_GAME",
  "data": {
    "gameId": "game-id-xyz"
  }
}
```

- **Sonuç:** Tüm oyunculara QUESTION_START event'i gönderilir.

3.4. SUBMIT_ANSWER - Cevap Gönder

- **Ne Zaman:** Oyuncu bir soruya cevap verdiğinde.
- **Gönderilecek Veri:**

```
JSON
{
  "type": "SUBMIT_ANSWER",
  "data": {
    "questionId": "question-uuid-123",
    "answerIndex": 2 // 0-3 arası (4 şık)
  }
}
```

- **Notlar:**
 - Her oyuncu bir soruya sadece 1 kez cevap verebilir.
 - Cevap kontrolü Redis'ten yapılır (çok hızlı).

3.5. NEXT_QUESTION - Sonraki Soru (Sadece Host)

- **Ne Zaman:** Host sonraki soruya geçmek istediğinde.
- **Gönderilecek Veri:**

```
JSON
```

```
{
  "type": "NEXT_QUESTION",
  "data": {
    "gameId": "game-id-xyz"
  }
}
```

- **Sonuç:**
 - Son soru ise GAME_OVER event'i gelir.
 - Değilse yeni QUESTION_START event'i gelir.

4. WebSocket Events (Server → Client)

Backend'in Frontend'e göndereceği olaylar.

4.1. JOIN_SUCCESS - Başarıyla Katıldın

- **Ne Zaman:** Lobiye başarıyla katıldıktan sonra (sadece o oyuncuya).

- **Alınacak Veri:**

```
JSON
{
  "type": "JOIN_SUCCESS",
  "data": {
    "status": "WAITING",
    "myNick": "Ahmet#1" // Çakışma varsa #1 eklenir
  }
}
```

- **Frontend Aksiyonu:** Lobi ekranına geç, Nickname'i sakla.

4.2. LOBBY_UPDATE - Lobi Sayacı (Sadece Host)

- **Ne Zaman:** Yeni oyuncu katıldığında (sadece host'a gönderilir).

- **Alınacak Veri:**

```
JSON
{
  "type": "LOBBY_UPDATE",
  "data": {
    "count": 45 // Şu anki oyuncu sayısı
  }
}
```

- **Frontend Aksiyonu:** Lobi sayacını güncelle.

4.3. FORCE_DISCONNECT - Atıldın

- **Ne Zaman:** Host seni attıysa.
- **Alınacak Veri:**

```
JSON
{
  "type": "FORCE_DISCONNECT",
  "data": {
    "reason": "Atıldınız"
  }
}
```

- **Frontend Aksiyonu:** Bağlantıyı kapat, ana ekrana dön veya hata göster.

4.4. QUESTION_START - Yeni Soru Geldi

- **Ne Zaman:** Oyun başladığında veya yeni soruya geçildiğinde (tüm oyunculara).
- **Alınacak Veri:**

```
JSON
{
  "type": "QUESTION_START",
  "data": {
    "questionIndex": 0,
    "text": "Türkiye'nin başkenti neresidir?",
    "mediaUrl": "https://example.com/image.jpg", // opsiyonel
    "options": [
      { "text": "Ankara", "color": "teal" },
      { "text": "İstanbul", "color": "pink" },
      { "text": "İzmir", "color": "purple" },
      { "text": "Bursa", "color": "orange" }
    ],
    "timeLimit": 30 // saniye
  }
}
```

- **ÖNEMLİ - Mod Farkı:**
 - **PERSONAL Mode:** options[].text dolu gelir.
 - **STAGE Mode:** options[].text boş string gelir (sadece renkler göster).
- **Frontend Aksiyonu:** Soruyu bas, Timer başlat, Şıkları göster (Moda göre).

4.5. ANSWER_RESULT - Cevap Sonucun

- **Ne Zaman:** Cevap gönderdikten sonra (sadece sana).
- **Alınacak Veri:**

```
JSON
{
  "type": "ANSWER_RESULT",
  "data": {
    "correct": true,    // Doğru mu?
    "earnedPoints": 1250, // Kazandığın puan
    "currentScore": 3750, // Toplam puanın
    "streak": 3        // Üst üste doğru sayısı
  }
}
```

- **Frontend Aksiyonu:** Doğru/yanlış animasyonu, kazanılan puan ve streak efekti göster.

4.6. SHOW_SCOREBOARD - Scoreboard Göster

- **Ne Zaman:** Soru bittiğinde (tüm oyunculara).
- **Alınacak Veri:**

```
JSON
{
  "type": "SHOW_SCOREBOARD",
  "data": {
    "topPlayers": [
      { "nickname": "Oyuncu1", "score": 5000 },
      { "nickname": "Oyuncu2", "score": 4500 },
      // ... ilk 5 oyuncu
    ]
  }
}
```

- **Frontend Aksiyonu:** Scoreboard ekranını göster, Top 5 listele.

4.7. GAME_OVER - Oyun Bitti

- **Ne Zaman:** Son soru bittiğinde (tüm oyunculara).
- **Alınacak Veri:**

```
JSON
{
  "type": "GAME_OVER",
  "data": {
    "winner": "Oyuncu1",
    "finalScores": [
      // ... tüm skorlar
    ]
  }
}
```

```
]
}
}
```

- **Frontend Aksiyonu:** Final ekranı, kazananı öne çıkar, konfetiler.

4.8. ERROR - Hata

- **Ne Zaman:** Bir şey ters giderse.
- **Alınacak Veri:**

```
JSON
{
  "type": "ERROR",
  "data": {
    "message": "Invalid PIN format"
  }
}
```

5. Oyun Akışı (Adım Adım Kod Örnekleri)

Adım 1: Oyun Oluştur (Host)

TypeScript

```
const response = await fetch('http://localhost:3000/games', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ quizId: 'quiz-uuid' })
});
const { pin, qrUrl, mode } = await response.json();
```

Adım 2: WebSocket Bağlan (Herkes)

TypeScript

```
const ws = new WebSocket('ws://localhost:3000/games/ws');
```

Adım 3: Lobiye Katıl (Herkes)

TypeScript

```
ws.send(JSON.stringify({  
  type: 'JOIN_ROOM',  
  data: { pin: '482910', nickname: 'Ahmet' }  
}));
```

Adım 4: Mesajları Dinle (Switch-Case)

TypeScript

```
ws.onmessage = (event) => {  
  const message = JSON.parse(event.data);  
  switch (message.type) {  
    case 'QUESTION_START':  
      // Soruyu göster  
      break;  
    case 'GAME_OVER':  
      // Bitir  
      break;  
    // ... diğer case'ler  
  }  
};
```

Adım 5: Oyunu Başlat (Host)

TypeScript

```
ws.send(JSON.stringify({
```

```
type: 'START_GAME',  
data: { gameId: 'game-id-xyz' }  
}});
```

Adım 6: Cevap Gönder (Oyuncu)

TypeScript

```
ws.send(JSON.stringify({  
  type: 'SUBMIT_ANSWER',  
  data: { questionId: currentQuestion.id, answerIndex: 2 }  
}));
```

Adım 7: Sonraki Soru (Host)

TypeScript

```
ws.send(JSON.stringify({  
  type: 'NEXT_QUESTION',  
  data: { gameId: 'game-id-xyz' }  
}));
```

6. TypeScript Type Definitions

(Bunu `types/game.ts` dosyasına kaydedebilirsiniz)

TypeScript

```
export type GameStatus = 'LOBBY' | 'ACTIVE' | 'FINISHED';  
export type GameMode = 'PERSONAL' | 'STAGE';
```

```
export type WebSocketEventType =  
  | 'JOIN_ROOM' | 'JOIN_SUCCESS' | 'LOBBY_UPDATE'  
  | 'KICK_PLAYER' | 'FORCE_DISCONNECT'  
  | 'START_GAME' | 'QUESTION_START' | 'SUBMIT_ANSWER'  
  | 'ANSWER_RESULT' | 'SHOW_SCOREBOARD' | 'NEXT_QUESTION'  
  | 'GAME_OVER' | 'ERROR';
```

```
// Client → Server Events
```

```
export interface JoinRoomEvent {  
  type: 'JOIN_ROOM';  
  data: { pin: string; nickname: string; };  
}  
export interface KickPlayerEvent {  
  type: 'KICK_PLAYER';  
  data: { socketId: string; ban: boolean; };  
}  
export interface StartGameEvent {  
  type: 'START_GAME';  
  data: { gameId: string; };  
}  
export interface SubmitAnswerEvent {  
  type: 'SUBMIT_ANSWER';  
  data: { questionId: string; answerIndex: number; };  
}  
export interface NextQuestionEvent {  
  type: 'NEXT_QUESTION';  
  data: { gameId: string; };  
}
```

```
// Server → Client Events
```

```
export interface JoinSuccessEvent {  
  type: 'JOIN_SUCCESS';  
  data: { status: 'WAITING'; myNick: string; };  
}  
export interface LobbyUpdateEvent {  
  type: 'LOBBY_UPDATE';  
  data: { count: number; };  
}  
export interface ForceDisconnectEvent {  
  type: 'FORCE_DISCONNECT';  
  data: { reason: string; };  
}  
export interface QuestionStartEvent {
```

```

    type: 'QUESTION_START';
    data: {
      questionIndex: number;
      text: string;
      mediaUrl?: string;
      options: Array<{ text: string; color: string }>;
      timeLimit: number;
    };
  }
}

export interface AnswerResultEvent {
  type: 'ANSWER_RESULT';
  data: {
    correct: boolean; earnedPoints: number; currentScore: number; streak: number;
  };
}

export interface LeaderboardEntry { nickname: string; score: number; }
export interface ShowScoreboardEvent {
  type: 'SHOW_SCOREBOARD';
  data: { topPlayers: LeaderboardEntry[]; };
}

export interface GameOverEvent {
  type: 'GAME_OVER';
  data: { winner: string; finalScores: LeaderboardEntry[]; };
}

export interface ErrorEvent {
  type: 'ERROR';
  data: { message: string; };
}

```

7. React/Next.js Hook Örneği

TypeScript

```

// hooks/useGameWebSocket.ts
import { useEffect, useRef, useState } from 'react';

export function useGameWebSocket(url: string) {

```

```

const ws = useRef<WebSocket | null>(null);
const [isConnected, setIsConnected] = useState(false);
const [lastMessage, setLastMessage] = useState<any>(null);

useEffect(() => {
  ws.current = new WebSocket(url);
  ws.current.onopen = () => setIsConnected(true);
  ws.current.onmessage = (event) => setLastMessage(JSON.parse(event.data));
  ws.current.onclose = () => setIsConnected(false);
  return () => { ws.current?.close(); };
}, [url]);

const send = (data: any) => {
  if (ws.current && isConnected) ws.current.send(JSON.stringify(data));
};

return { isConnected, lastMessage, send };
}

```

8. Test Etme

Browser Console ile:

JavaScript

```

const ws = new WebSocket('ws://localhost:3000/games/ws');
ws.onmessage = (e) => console.log(JSON.parse(e.data));
ws.send(JSON.stringify({type:"JOIN_ROOM",data:{pin:"482910",nickname:"Test"}}));

```

wscat ile (Terminal):

Bash

```
wscat -c ws://localhost:3000/games/ws
```

```
> {"type":"JOIN_ROOM","data":{"pin":"482910","nickname":"TestUser"}}
```

9. Önemli Notlar

1. **STAGE vs PERSONAL Mod:**
 - **STAGE:** Soru ekranda, mobilde sadece renkli butonlar.
 - **PERSONAL:** Her oyuncu kendi ekranında soruyu görür.
2. **Tek Cevap:** Her oyuncu bir soruya sadece 1 kez cevap verebilir.
3. **Otomatik Scoreboard:** Tüm oyuncular cevap verdiğiinde otomatik gösterilir, Host'un tetiklemesine gerek yoktur.
4. **Bağlantı Kopması:** Reconnect mekanizması şimdilik yoktur, kopan oyundan atılır.
5. **IP Ban:** Banlanan kişi farklı nickname ile de olsa aynı IP'den tekrar giremez.